

1. 전처리

1) Sample Rate: 22050 Hz

2) Channels: Mono

3) Bit Depth: 32-float point

4) Duration: 4초, repeat padding

5) Normalization: 각 오디오 파일에 대해 40개의 MFCC(Mel-Frequency Cepstral Coefficients)를 추출한 후, 각 계수의 평균(mean)과 표준편차(std)를 계산하여 총 80차원의 통계적 특징(feature)을 생성하였다.

추출된 MFCC 특징에 대해 Train fold(1~8)만을 이용하여 평균과 표준편차를 계산한 후 Standardization을 수행하였다. 이후 동일한 변환을 Validation fold(9)와 Test fold(10)에 적용하였다.

(아래 스크린샷은 1)~4) 포함, 5) Normalization 미포함)

```
(base) PS C:\Users\sp9gh\Downloads> python preprocess.py
500개 처리 완료
1000개 처리 완료
1500개 처리 완료
2000개 처리 완료
2500개 처리 완료
3000개 처리 완료
3500개 처리 완료
4000개 처리 완료
4500개 처리 완료
5000개 처리 완료
5500개 처리 완료
6000개 처리 완료
6500개 처리 완료
7000개 처리 완료
7500개 처리 완료
8000개 처리 완료
8500개 처리 완료
X shape: (8732, 80)
y shape: (8732,)
folds shape: (8732,)
저장 완료!
(base) PS C:\Users\sp9gh\Downloads> |
```

• 코딩 (preprocess.py)

```
import os
import numpy as np
import pandas as pd
import librosa
```

```
DATA_DIR = r"C:\Users\sp9gh\Downloads\UrbanSound8K\UrbanSound8K"
```

```
CSV_PATH = os.path.join(DATA_DIR, "metadata", "UrbanSound8K.csv")
```

```
AUDIO_DIR = os.path.join(DATA_DIR, "audio")
```

```
TARGET_SR = 22050
TARGET_DURATION = 4
TARGET_LENGTH = TARGET_SR * TARGET_DURATION
```

```
def preprocess_audio(file_path):
    y, sr = librosa.load(file_path, sr=TARGET_SR, mono=True)
    y = y.astype(np.float32)

    if len(y) > TARGET_LENGTH:
        y = y[:TARGET_LENGTH]

    elif len(y) < TARGET_LENGTH:
        repeat_count = int(np.ceil(TARGET_LENGTH / len(y)))
        y = np.tile(y, repeat_count)[:TARGET_LENGTH]

    return y
```

```
def extract_mfcc_features(y, sr=TARGET_SR, n_mfcc=40):
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)

    feature = np.concatenate([
        mfcc.mean(axis=1),
        mfcc.std(axis=1)
    ])

    return feature
```

```
df = pd.read_csv(CSV_PATH)
```

```
X = []
y = []
folds = []
class_names = []
```

```
for idx, row in df.iterrows():
    file_path = os.path.join(
        AUDIO_DIR,
        f"fold{row['fold']}",
        row["slice_file_name"]
    )

    try:
        waveform = preprocess_audio(file_path)
        feature = extract_mfcc_features(waveform)

        X.append(feature)
        y.append(row["classID"])
        folds.append(row["fold"])
        class_names.append(row["class"])
```

```
except Exception as e:
    print("오류 발생:", file_path)
    print(e)

if (idx + 1) % 500 == 0:
    print(f'{idx + 1}개 처리 완료')
```

```
X = np.array(X)
y = np.array(y)
folds = np.array(folds)
class_names = np.array(class_names)

print("X shape:", X.shape)
print("y shape:", y.shape)
print("folds shape:", folds.shape)

np.save("X_mfcc.npy", X)
np.save("y.npy", y)
np.save("folds.npy", folds)
np.save("class_names.npy", class_names)

print("저장 완료!")
```

2. 모델 설명 및 feature

1) XGBoost

XGBoost는 여러 개의 **decision tree**를 순차적으로 학습시키는 **gradient boosting** 기반 모델이다. 이전 **tree**가 잘 맞지 못한 **sample**에 더 집중하면서 성능을 개선한다. 비선형 관계를 잘 포착하고, 다양한 **acoustic feature**를 함께 사용할 때 성능이 좋은 편이다.

2) SVM

SVM은 **class**를 구분하는 최적의 경계면을 찾는 분류 모델이다. 특히 **RBF kernel**을 사용하면 **feature** 간 비선형 관계도 학습할 수 있다. 다만 **feature scale**에 민감하므로 **standardization**이 중요하다.

3) feature

각 오디오 파일에 대해 40개의 MFCC(Mel-Frequency Cepstral Coefficients)를 추출한 후, 각 계수의 평균(mean)과 표준편차(std)를 계산하였다. 이를 통해 파일당 총 80차원의 MFCC 통계 특징(MFCC statistical features)을 생성하였다.

Model	Input Feature
SVM	MFCC statistical features (40 mean + 40 std = 80 features)
XGBoost	MFCC statistical features (40 mean + 40 std = 80 features)

```
(base) PS C:\Users\sp9gh\Downloads> pip show xgboost
WARNING: Package(s) not found: xgboost
(base) PS C:\Users\sp9gh\Downloads> pip install xgboost
Collecting xgboost
  Downloading xgboost-3.2.0-py3-none-win_amd64.whl.metadata (2.1 kB)
Requirement already satisfied: numpy in c:\users\sp9gh\anaconda3\lib\site-packages (from xgboost) (2.3.5)
Requirement already satisfied: scipy in c:\users\sp9gh\anaconda3\lib\site-packages (from xgboost) (1.16.3)
Downloading xgboost-3.2.0-py3-none-win_amd64.whl (101.7 MB)
 101.7/101.7 MB 7.3 MB/s 0:00:13
Installing collected packages: xgboost
Successfully installed xgboost-3.2.0
(base) PS C:\Users\sp9gh\Downloads> python -c "from sklearn.svm import SVC; print('SVM 사용 가능')"
SVM 사용 가능
(base) PS C:\Users\sp9gh\Downloads> |
```

- 코딩 (train_models.py)

```
import numpy as np
import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

```
from xgboost import XGBClassifier
```

```
# =====
# 데이터 불러오기
# =====
```

```
X = np.load("X_mfcc.npy")
y = np.load("y.npy")
folds = np.load("folds.npy")
class_names = np.load("class_names.npy", allow_pickle=True)
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)
print("folds shape:", folds.shape)
```

```
# =====
# Train / Validation / Test 분리
# =====
```

```
train_mask = folds <= 8
val_mask = folds == 9
test_mask = folds == 10
```

```
X_train = X[train_mask]
y_train = y[train_mask]
```

```
X_val = X[val_mask]
y_val = y[val_mask]
```

```
X_test = X[test_mask]
y_test = y[test_mask]
```

```
print("Train:", X_train.shape)
print("Validation:", X_val.shape)
print("Test:", X_test.shape)
```

```
# =====
# Train 기준 Standardization
# =====
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

```
print("Standardization 완료")
```

```
# =====
# 평가 함수
# =====
```

```
def evaluate_model(model_name, y_true, y_pred):
    results = {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Balanced Accuracy": balanced_accuracy_score(y_true, y_pred),
        "Macro Precision": precision_score(y_true, y_pred, average="macro", zero_division=0),
        "Macro Recall": recall_score(y_true, y_pred, average="macro", zero_division=0),
        "Macro F1": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "Weighted Precision": precision_score(y_true, y_pred, average="weighted", zero_division=0),
        "Weighted Recall": recall_score(y_true, y_pred, average="weighted", zero_division=0),
        "Weighted F1": f1_score(y_true, y_pred, average="weighted", zero_division=0),
    }
    return results
```

```
# =====
# SVM 학습
# =====
```

```
svm_model = SVC(
    kernel="rbf",
```

```

C=10,
gamma="scale",
class_weight="balanced",
random_state=42
)

print("SVM 학습 시작")
svm_model.fit(X_train_scaled, y_train)
print("SVM 학습 완료")

svm_pred = svm_model.predict(X_test_scaled)

# =====
# XGBoost 학습
# =====

xgb_model = XGBClassifier(
    n_estimators=300,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="multi:softmax",
    num_class=10,
    eval_metric="mlogloss",
    random_state=42
)

print("XGBoost 학습 시작")
xgb_model.fit(X_train_scaled, y_train)
print("XGBoost 학습 완료")

xgb_pred = xgb_model.predict(X_test_scaled)

# =====
# 전체 평가 결과표
# =====

results = [
    evaluate_model("SVM", y_test, svm_pred),
    evaluate_model("XGBoost", y_test, xgb_pred)
]

results_df = pd.DataFrame(results)
print("\n=== Overall Results ===")
print(results_df)

results_df.to_csv("overall_results.csv", index=False, encoding="utf-8-sig")

# =====

```

```
# Per-class 평가 결과
```

```
# =====
```

```
target_names = [  
    "air_conditioner",  
    "car_horn",  
    "children_playing",  
    "dog_bark",  
    "drilling",  
    "engine_idling",  
    "gun_shot",  
    "jackhammer",  
    "siren",  
    "street_music"  
]
```

```
]
```

```
svm_report = classification_report(  
    y_test,  
    svm_pred,  
    target_names=target_names,  
    output_dict=True,  
    zero_division=0  
)
```

```
xgb_report = classification_report(  
    y_test,  
    xgb_pred,  
    target_names=target_names,  
    output_dict=True,  
    zero_division=0  
)
```

```
svm_report_df = pd.DataFrame(svm_report).transpose()  
xgb_report_df = pd.DataFrame(xgb_report).transpose()
```

```
svm_report_df.to_csv("svm_per_class_results.csv", encoding="utf-8-sig")  
xgb_report_df.to_csv("xgb_per_class_results.csv", encoding="utf-8-sig")
```

```
print("\n=== SVM Per-class Results ===")  
print(svm_report_df)
```

```
print("\n=== XGBoost Per-class Results ===")  
print(xgb_report_df)
```

```
# =====
```

```
# Confusion Matrix 저장
```

```
# =====
```

```
svm_cm = confusion_matrix(y_test, svm_pred)  
xgb_cm = confusion_matrix(y_test, xgb_pred)
```

```
svm_cm_df = pd.DataFrame(  

```

```

svm_cm,
index=target_names,
columns=target_names
)

xgb_cm_df = pd.DataFrame(
    xgb_cm,
    index=target_names,
    columns=target_names
)

svm_cm_df.to_csv("svm_confusion_matrix.csv", encoding="utf-8-sig")
xgb_cm_df.to_csv("xgb_confusion_matrix.csv", encoding="utf-8-sig")

print("\n저장 완료:")
print("overall_results.csv")
print("svm_per_class_results.csv")
print("xgb_per_class_results.csv")
print("svm_confusion_matrix.csv")
print("xgb_confusion_matrix.csv")

```

3. 결과

```

(base) PS C:\Users\sp9gh\Downloads> python train_models.py
X shape: (8732, 80)
y shape: (8732,)
folds shape: (8732,)
Train: (7079, 80)
Validation: (816, 80)
Test: (837, 80)
Standardization 완료
SVM 학습 시작
SVM 학습 완료
XGBoost 학습 시작
XGBoost 학습 완료

=== Overall Results ===
   Model  Accuracy  Balanced Accuracy  Macro Precision  Macro Recall  Macro F1  Weighted Precision  Weighted Recall  Weighted F1
0      SVM    0.702509         0.722153      0.732377      0.722153    0.723635         0.711003         0.702509      0.702494
1  XGBoost    0.728793         0.734558      0.776564      0.734558    0.746983         0.749381         0.728793      0.730958

=== SVM Per-class Results ===
               precision      recall  f1-score   support
air_conditioner    0.790698    0.680000    0.731183    100.000000
car_horn           0.812500    0.787879    0.800000     33.000000
children_playing   0.689655    0.800000    0.740741    100.000000
dog_bark           0.604478    0.810000    0.692308    100.000000
drilling           0.555556    0.600000    0.576923    100.000000
engine_idling      0.888889    0.774194    0.827586     93.000000
gun_shot           0.937500    0.937500    0.937500     32.000000
jackhammer         0.538462    0.437500    0.482759     96.000000
siren              0.629630    0.614458    0.621951     83.000000
street_music       0.876404    0.780000    0.825397    100.000000
accuracy           0.702509    0.702509    0.702509     0.702509
macro avg          0.732377    0.722153    0.723635    837.000000
weighted avg       0.711003    0.702509    0.702494    837.000000

```

=== XGBoost Per-class Results ===

	precision	recall	f1-score	support
air_conditioner	0.904762	0.760000	0.826087	100.000000
car_horn	1.000000	0.727273	0.842105	33.000000
children_playing	0.540000	0.810000	0.648000	100.000000
dog_bark	0.740741	0.800000	0.769231	100.000000
drilling	0.584746	0.690000	0.633028	100.000000
engine_idling	0.925926	0.806452	0.862069	93.000000
gun_shot	0.903226	0.875000	0.888889	32.000000
jackhammer	0.632911	0.520833	0.571429	96.000000
siren	0.700000	0.506024	0.587413	83.000000
street_music	0.833333	0.850000	0.841584	100.000000
accuracy	0.728793	0.728793	0.728793	0.728793
macro avg	0.776564	0.734558	0.746983	837.000000
weighted avg	0.749381	0.728793	0.730958	837.000000

저장 완료 :

overall_results.csv

svm_per_class_results.csv

xgb_per_class_results.csv

svm_confusion_matrix.csv

xgb_confusion_matrix.csv

(base) PS C:\Users\sp9gh\Downloads>

```
(base) PS C:\Users\sp9gh\Downloads> type svm_confusion_matrix.csv
,air_conditioner,car_horn,children_playing,dog_bark,drilling,engine_idling,gun_shot,jackhammer,siren,street_music
air_conditioner,68,0,0,5,1,8,0,17,1,0
car_horn,1,26,0,0,0,0,0,0,1,0,5
children_playing,1,0,80,12,0,0,0,0,7,0
dog_bark,0,4,8,81,1,1,0,0,0,5
drilling,0,0,1,4,60,0,2,18,14,1
engine_idling,9,1,0,5,0,72,0,0,6,0
gun_shot,0,0,0,2,0,0,30,0,0,0
jackhammer,6,0,2,1,45,0,0,42,0,0
siren,0,0,8,24,0,0,0,0,51,0
street_music,1,1,17,0,1,0,0,0,2,78
(base) PS C:\Users\sp9gh\Downloads> type xgb_confusion_matrix.csv
,air_conditioner,car_horn,children_playing,dog_bark,drilling,engine_idling,gun_shot,jackhammer,siren,street_music
air_conditioner,76,0,0,3,2,4,0,15,0,0
car_horn,2,24,0,0,0,0,0,1,0,6
children_playing,0,0,81,10,0,2,0,0,4,3
dog_bark,0,0,11,80,2,0,1,0,1,5
drilling,0,0,5,3,69,0,2,10,9,2
engine_idling,5,0,5,1,0,75,0,3,3,1
gun_shot,0,0,0,4,0,0,28,0,0,0
jackhammer,1,0,1,0,44,0,0,50,0,0
siren,0,0,34,7,0,0,0,0,42,0
street_music,0,0,13,0,1,0,0,0,1,85
(base) PS C:\Users\sp9gh\Downloads> |
```

1) 전체 성능 비교

평가 지표	SVM	XGBoost
Accuracy	0.703	0.729
Balanced Accuracy	0.722	0.735

Macro Precision	0.732	0.777
Macro Recall	0.722	0.735
Macro F1	0.724	0.747
Weighted Precision	0.711	0.749
Weighted Recall	0.703	0.729
Weighted F1	0.702	0.731

• 결과 해석

MFCC 통계 특징을 이용하여 환경음을 분류한 결과, XGBoost가 SVM보다 모든 평가 지표에서 우수한 성능을 보였다. Accuracy는 SVM이 70.3%, XGBoost가 72.9%로 나타났으며, Macro F1-score 역시 SVM의 0.724보다 XGBoost의 0.747이 더 높았다. 특히 Macro Precision과 Weighted F1에서 차이가 두드러졌으며, 이는 XGBoost가 다양한 클래스에 대해 보다 안정적이고 균형 잡힌 분류 성능을 보였음을 의미한다. 따라서 본 실험에서는 XGBoost가 UrbanSound8K 환경음 분류에 더 적합한 모델로 판단된다.

2) SVM Per-Class 결과

Class	Precision	Recall	F1-score
air_conditioner	0.791	0.680	0.731
car_horn	0.813	0.788	0.800
children_playing	0.690	0.800	0.741
dog_bark	0.604	0.810	0.692
drilling	0.556	0.600	0.577
engine_idling	0.889	0.774	0.828
gun_shot	0.938	0.938	0.938
jackhammer	0.538	0.438	0.483
siren	0.630	0.614	0.622
street_music	0.876	0.780	0.825

• 결과 해석

SVM에서는 gun_shot 클래스가 Precision, Recall, F1-score 모두 0.938로 가장 우수한 성능을 보였다. 이는 총소리가 다른 환경음에 비해 독특한 음향 특성을 가지기 때문으로 해석된다. 반면 jackhammer는 F1-score가 0.483으로 가장 낮은 성능을 보였으며, drilling과의 혼동이 빈번하게 발생하였다. 또한 dog_bark는 Recall은 높지만 Precision이 상대적으로 낮아 다른 클래스가 dog_bark로 잘못 분류되는 경우가 많았음을 확인할 수 있었다. 전반적으로 SVM은 총격음이나 특징이 뚜렷한 소리에서는 높은 성능을 보였으나, 유사한 기계음이나 배경음이 포함된 클래스에서는 분류 성능이 다소 낮게 나타났다.

3) XGBoost Per-Class 결과

Class	Precision	Recall	F1-score
air_conditioner	0.905	0.760	0.826
car_horn	1.000	0.727	0.842
children_playing	0.540	0.810	0.648
dog_bark	0.741	0.800	0.769
drilling	0.585	0.690	0.633
engine_idling	0.926	0.806	0.862
gun_shot	0.903	0.875	0.889
jackhammer	0.633	0.521	0.571
siren	0.700	0.506	0.587
street_music	0.833	0.850	0.842

• 결과 해석

XGBoost에서는 engine_idling(F1=0.862), car_horn(F1=0.842), street_music(F1=0.842) 등이 높은 성능을 보였다. 특히 car_horn은 Precision이 1.000으로 나타나 car_horn으로 예측된 샘플은 모두 실제 car_horn이었음을 의미한다. 반면 children_playing은 Recall은 높았지만 Precision이 0.540으로 낮아 다른 클래스가 children_playing으로 잘못 분류되는 경우가 많았다. 또한 jackhammer와 siren은 상대적으로 낮은 F1-score를 보여 여전히 분류가 어려운 클래스로 나타났다. 전반적으로 XGBoost는 대부분의 클래스에서 SVM보다 높은 Precision과 F1-score를 기록하며 더 우수한 분류 성능을 보였다.

4) Confusion Matrix 해석

Confusion Matrix 분석 결과 두 모델 모두 drilling과 jackhammer를 자주 혼동하는 것으로 나타났다. SVM에서는 jackhammer의 45개 샘플이 drilling으로 잘못 분류되었으며, XGBoost에서도 44개의 jackhammer 샘플이 drilling으로 분류되었다. 이는 두 클래스가 모두 공사장 환경에서 발생하는 기계음으로 유사한 음향 특성을 가지기 때문으로 해석된다.

또한 XGBoost에서는 siren 클래스의 34개 샘플이 children_playing으로 분류되는 현상이 관찰되었다. 이는 야외 환경에서 녹음된 소리들이 공통적인 배경 소음을 포함하고 있어 일부 주파수 특성이 유사하게 나타났기 때문으로 볼 수 있다. SVM에서는 siren이 dog_bark로 분류된 경우가 24건 발생하여 두 클래스 간의 혼동도 확인되었다.

반면 gun_shot은 두 모델 모두에서 높은 정확도로 분류되었으며, 다른 클래스와의 혼동이 매우 적었다. 이는 총소리가 매우 짧고 강한 충격음을 가지는 특수한 음향 패턴을 가지고 있기 때문으로 판단된다.

• 결론

MFCC 통계 특징(mean, standard deviation)을 이용하여 UrbanSound8K 데이터셋의 환경음을 분류한 결과, XGBoost가 SVM보다 높은 Accuracy, Macro F1-score, Weighted F1-score를 기록하여 전반적으로 더 우수한 성능을 보였다. Confusion Matrix 분석 결과 drilling과 jackhammer, siren과 children_playing 등 음향적 특성이 유사한 클래스 간에 주된 오분류가 발생하였으며, 이는 환경음 분류 문제의 주요 한계로 확인되었다. 따라서 본 실험에서는 XGBoost가 환경음 분류에 보다 적합한 모델로 평가되었다.